

Object Localization, Classification and Detection

Prepared By
Dr. Khaled Mostafa ElSayed
kmelsayed@zewailcity.edu.eg

Instructor:

Dr. Khaled Mostafa ElSayed
kmelsayed@zewailcity.edu.eg



جامعة العلوم والتكنولوجيا
University of Science and Technology

Classification, Localization , Detection

2

Object Classification

- Single Object
- Define to which class this object belongs to

Object Localization and Classification

- Single Object
- Define to which class this object belongs to
- Draw a bounding Box around the object location

Object Detection

- Multiple Objects
- Draw a bounding Box around the objects locations

Examples of Objects Classification, Localization, and Detection

3



Person



Car

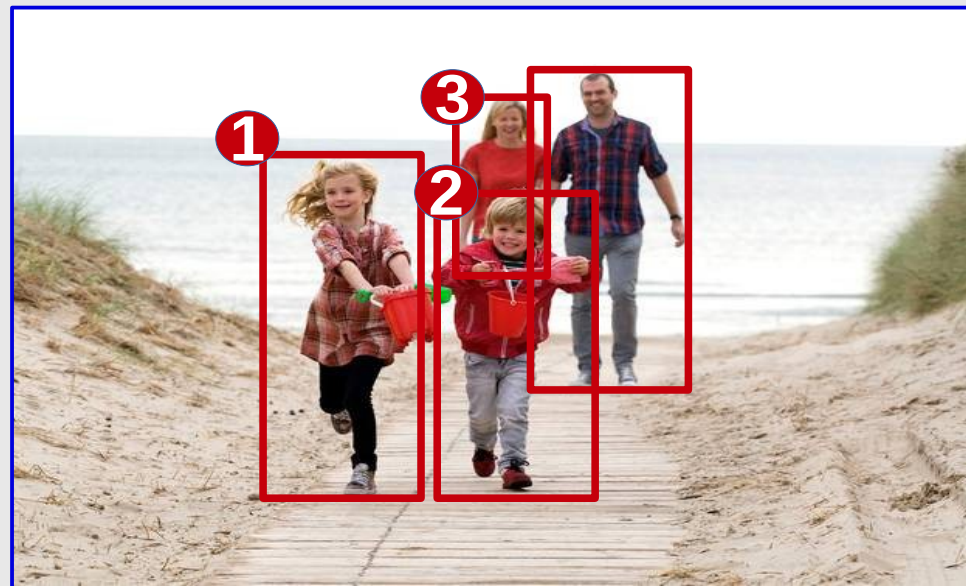


4

Apple

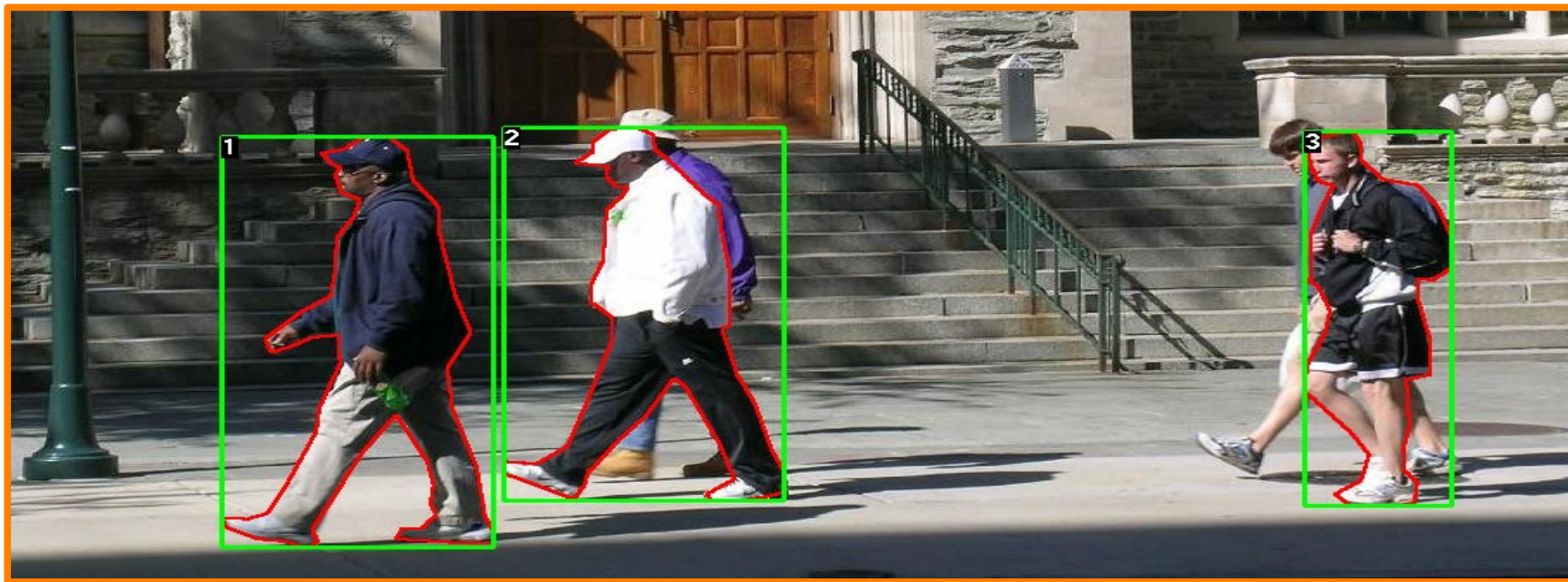


Person
X,Y,W,H



Detect Learned Objects ONLY

Localization vs Segmentation



[1] Classification

5



[2] Localization

6

(0,0)



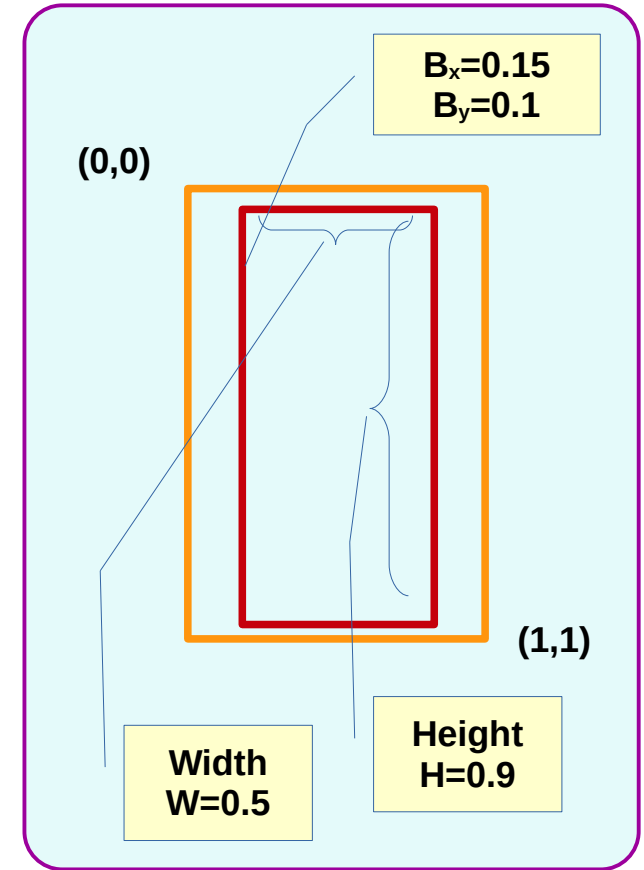
(1,1)

(0,0)



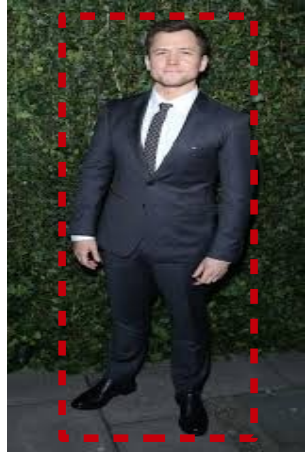
(1,1)

Normalized Image Width and Height (1,1)



Classification

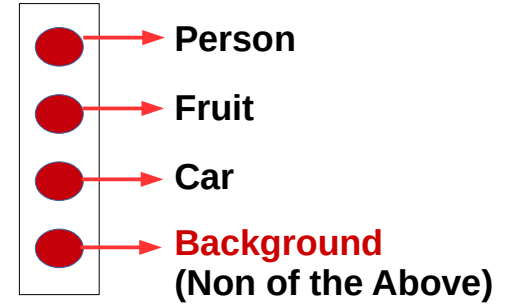
7



CNN



Soft max Layer

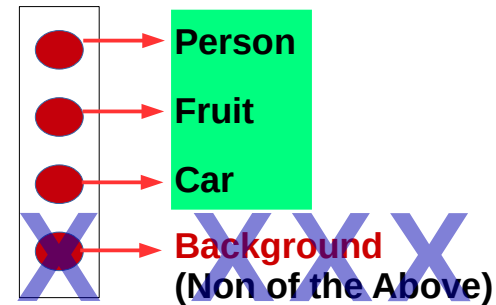


Localization and Classification

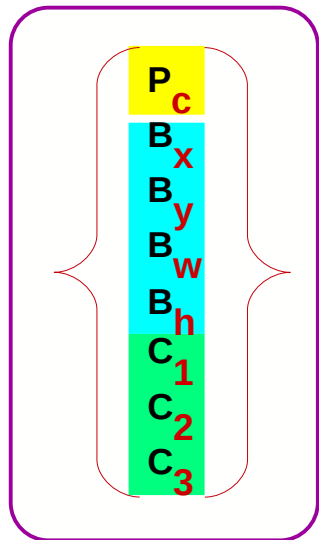
8



Soft max Layer



Output
Y

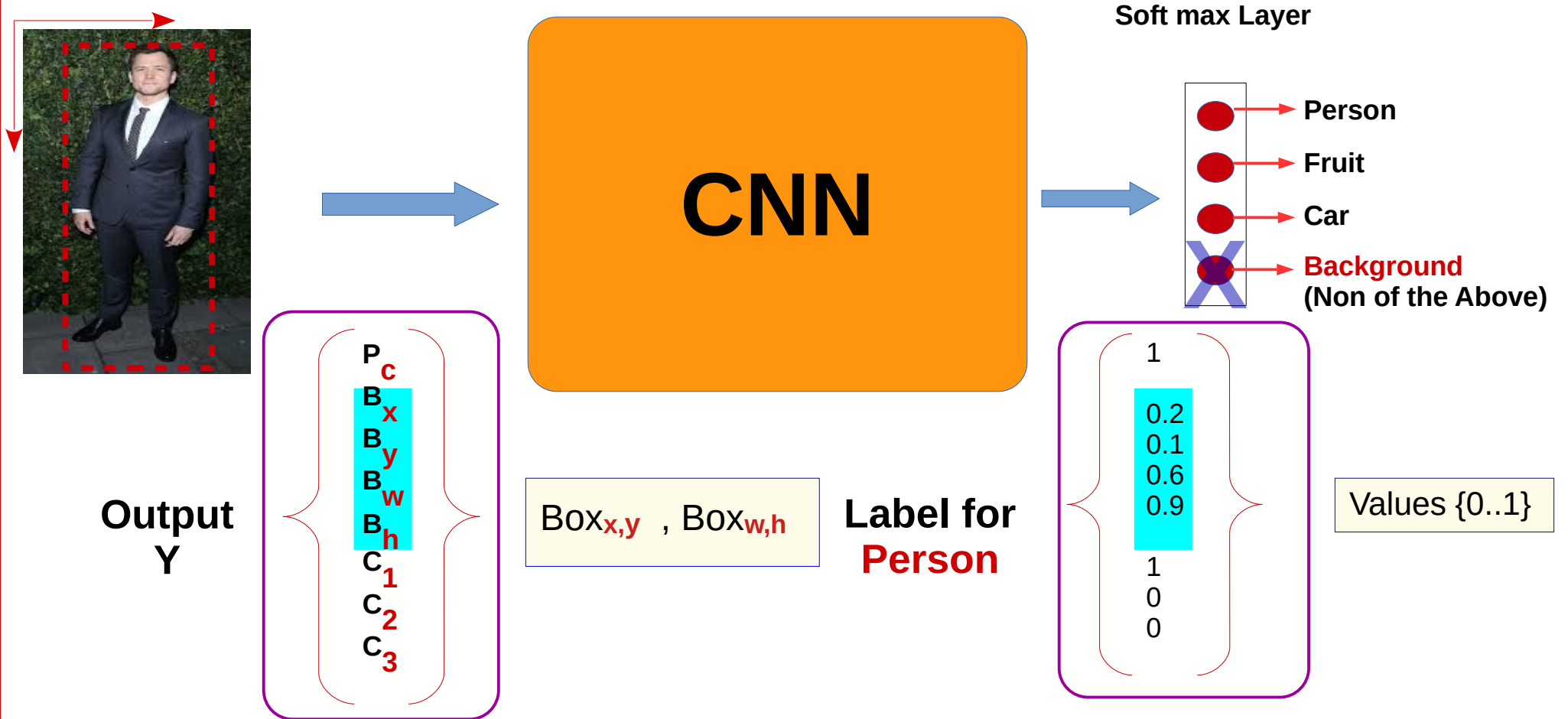


P_c is the Probability that there exist one of the Classes

IF $P_c = 0 \Rightarrow$ All Values are Don't Cate

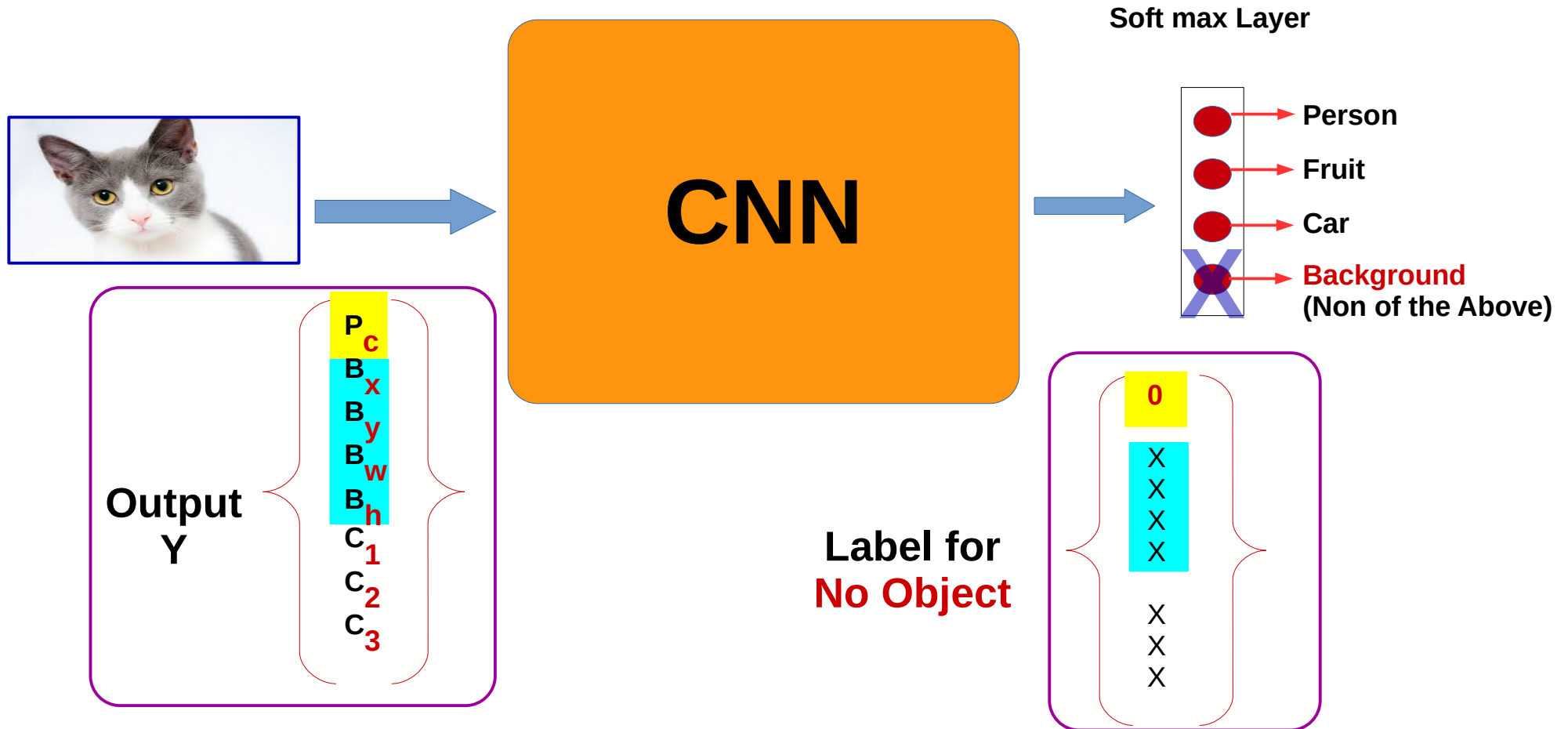
Localization and Classification

9



Localization and Classification

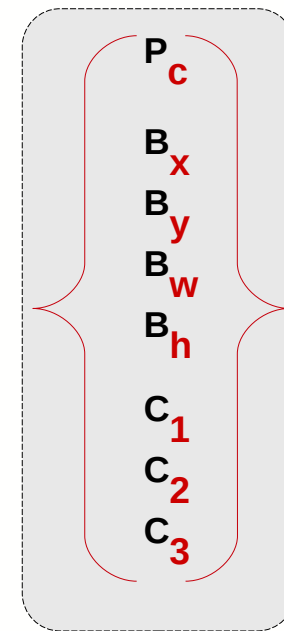
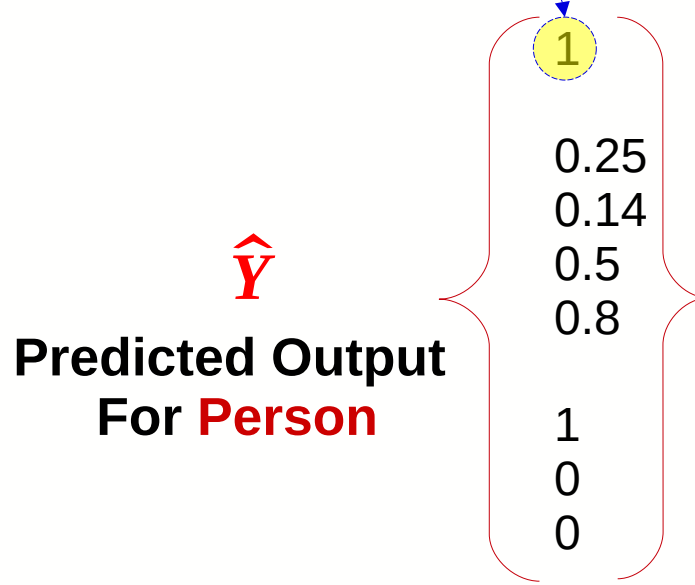
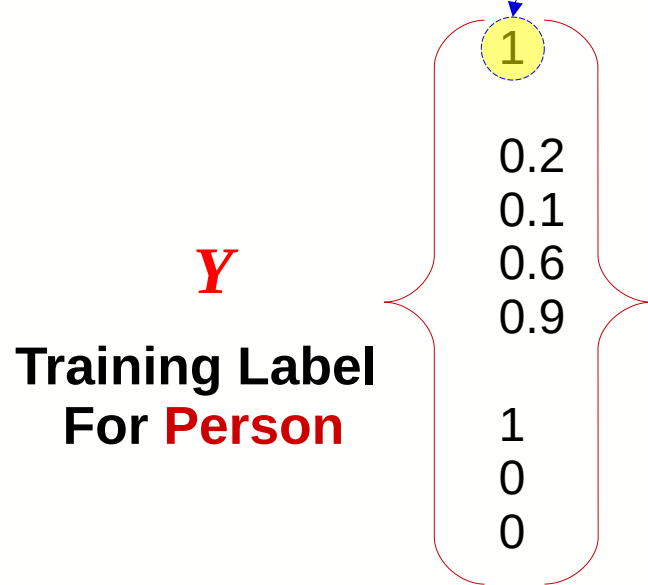
10



Localization and Classification Loss Calculations

In Case of existence of Object ($Y_1 = P_c = 1$)

Object Exists



$$Loss = (y_1 - \hat{y}_1)^2 + (y_2 - \hat{y}_2)^2 + (y_3 - \hat{y}_3)^2 + \dots + (y_8 - \hat{y}_8)^2 +$$

Localization and Classification Loss Calculations

In Case of existence of Object ($Y_1 = P_c = 0$)

No Object

Y
Training Label
For **Background**

0

x
x
x
x

x
x
x

$\hat{Y}$
Predicted Output
For **Person**

0.2

0.25
0.14
0.5
0.8

1
0
0

P_c

B_x

B_y

B_w

B_h

C_1

C_2

C_3

$$Loss = (y_1 - \hat{y}_1)^2$$

Other values are **Don't Care**

Localization and Classification Loss Calculations

In Case of existence of Object ($Y_1 = P_c = 1$)

Y
Training Label
For **Person**

1
0.2
0.1
0.6
0.9
1
0
0

$\hat{Y}$
Predicted Output
For **Person**

1
0.25
0.14
0.5
0.8
1
0
0

P_c
 B_x
 B_y
 B_w
 B_h
 C_1
 C_2
 C_3

Squared Error Loss Function for ALL Values

$$Loss = (y_1 - \hat{y}_1)^2 + (y_2 - \hat{y}_2)^2 + (y_3 - \hat{y}_3)^2 + \dots + (y_8 - \hat{y}_8)^2 +$$

Other Loss Functions that May be used:

Log Error Loss

for softmax classification ($C_1, C_2, C_3, \dots$)

Logistic Regression Loss

for Binary classification (P_c)

Object Detection

(Detection of **Multiple** Objects)

Detection of Multiple Objects

[1] Sliding Window

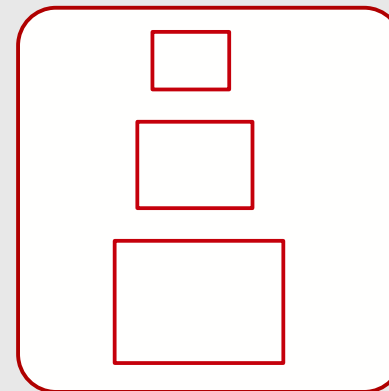
[2] You Only Look Once (YOLO)

[3] Regional CNN (RCNN) Family
(RCNN – Fast RCNN – Faster RCNN)

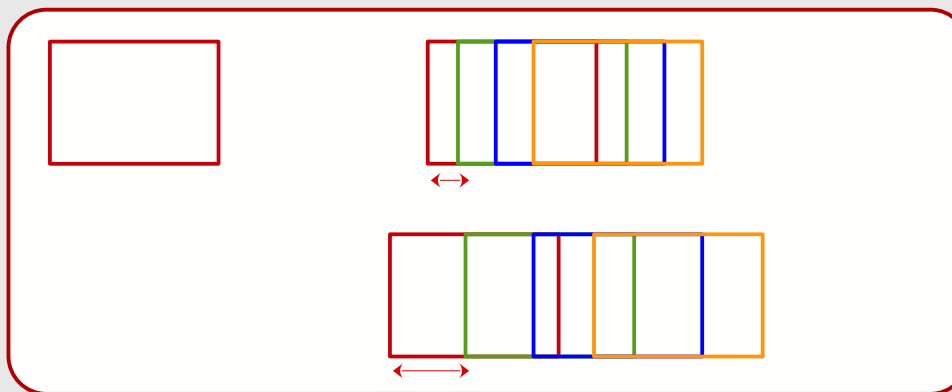
[1] Sliding Window Technique

Sliding Window Technique

Use Different Window Sizes



Select the required Stride (precision)

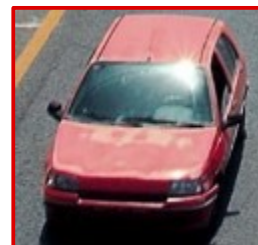
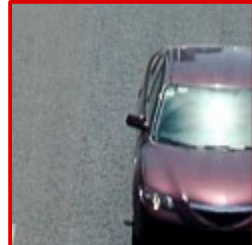
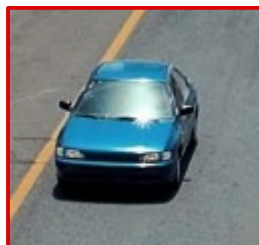
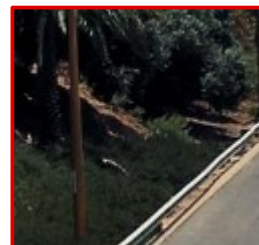
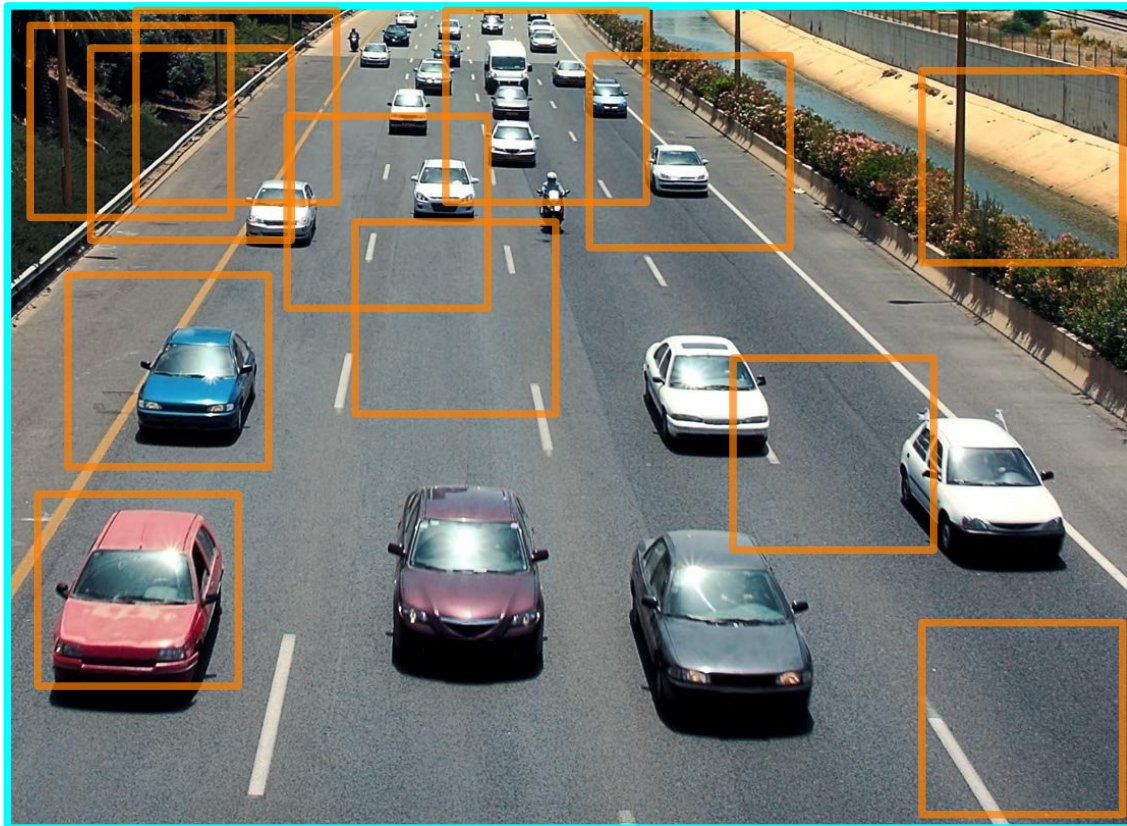




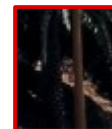
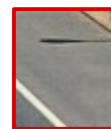
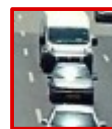
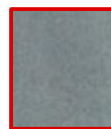
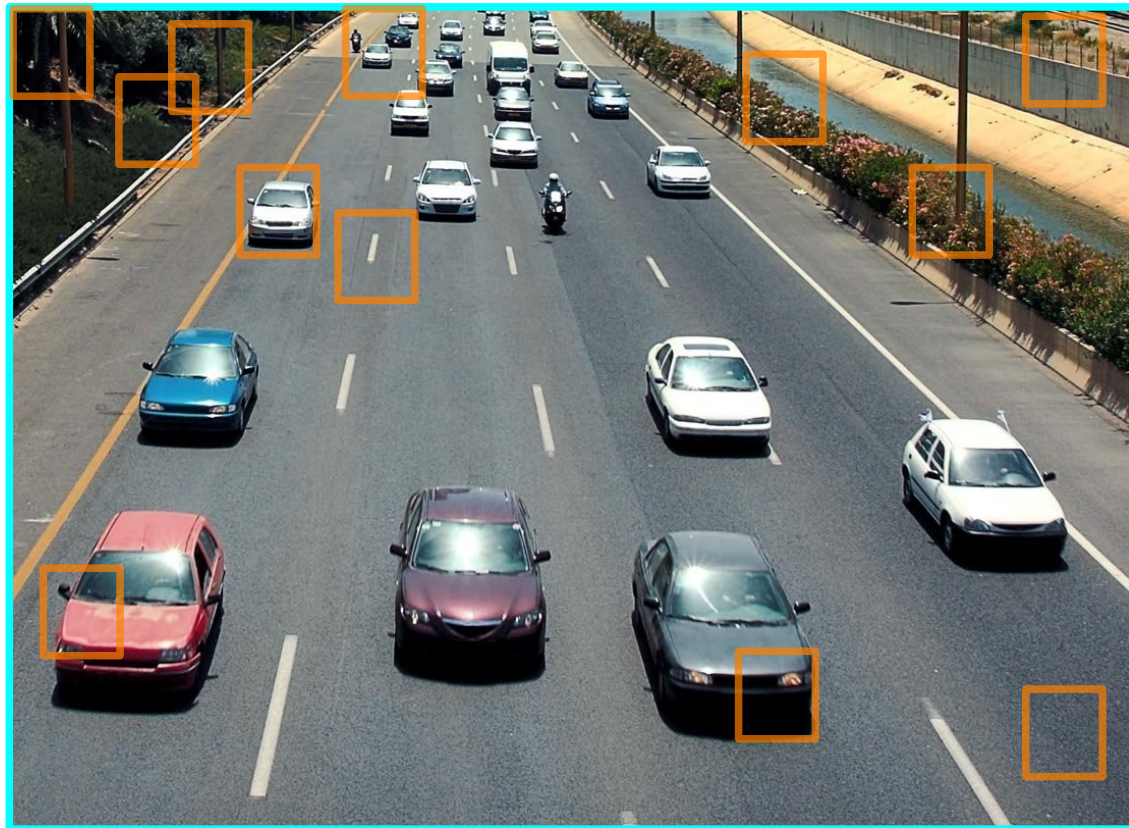
Input Image

Scan using **MULTIPLE** Windows sizes

Window
of Size 1

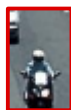
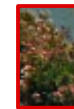
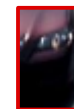
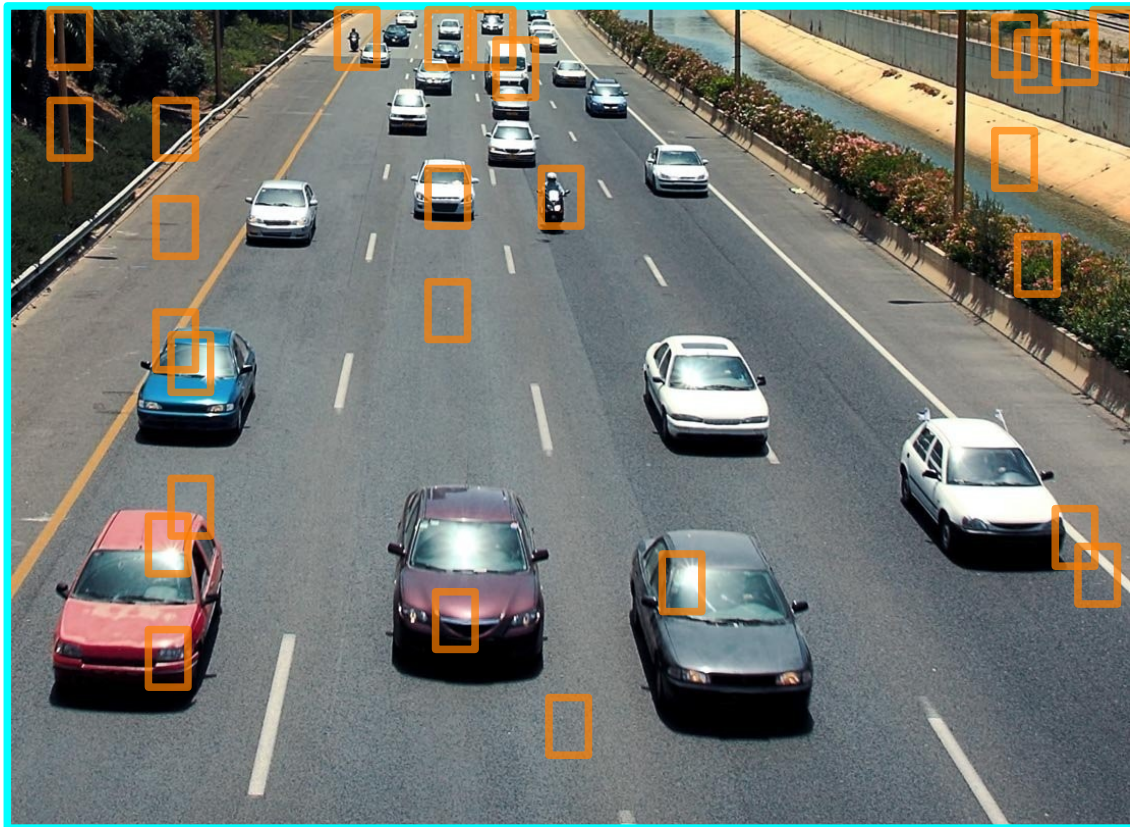


Window
of Size 2

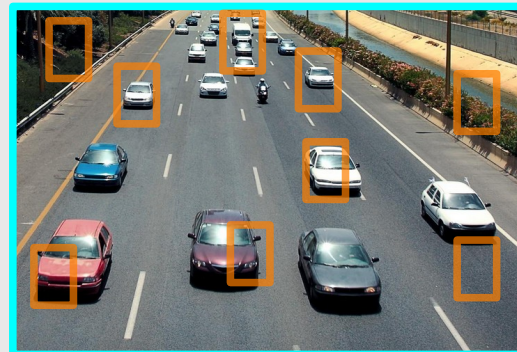
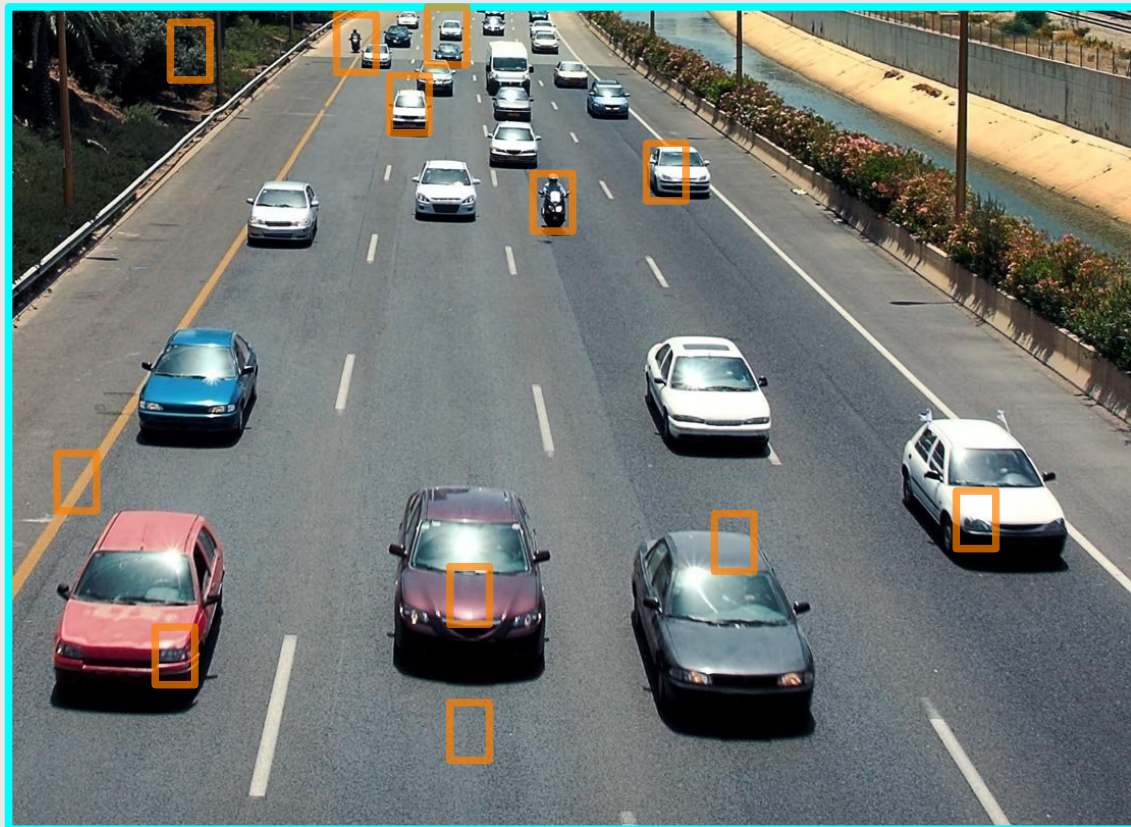




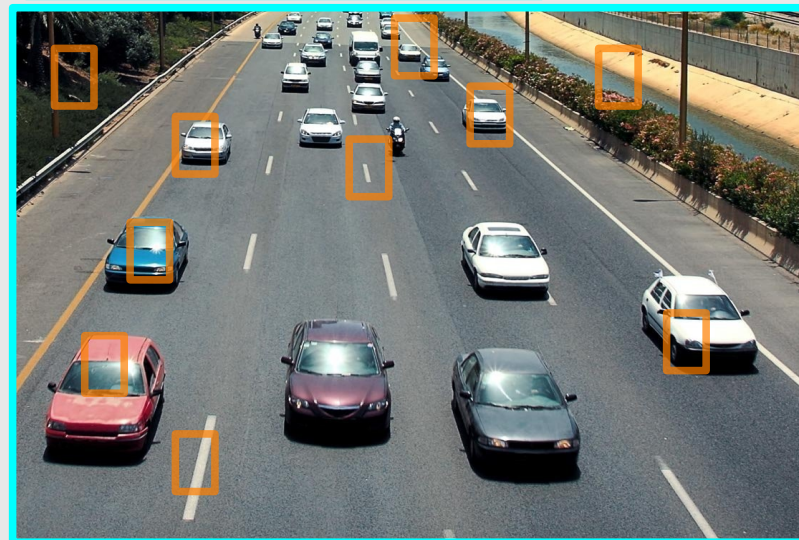
Window
of Size 3



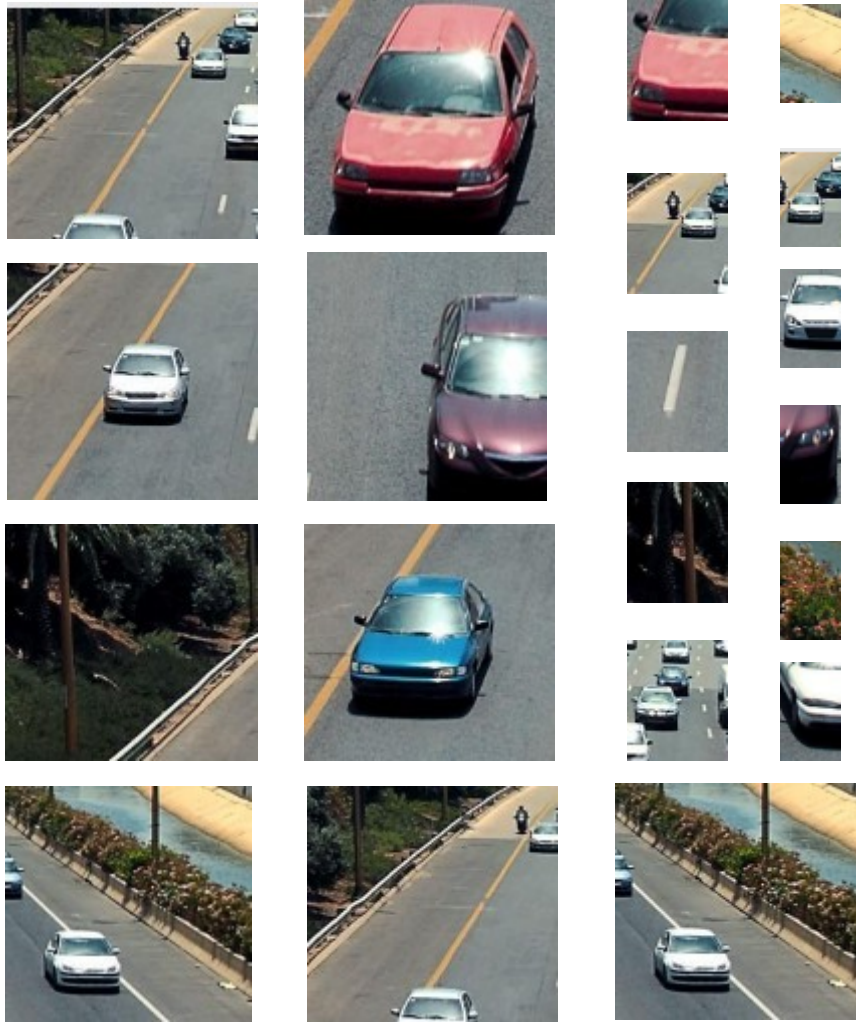
Alternatively, One Window size, Multiple Image Scales



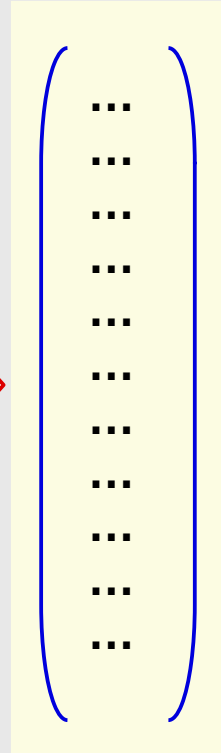
Window
Size



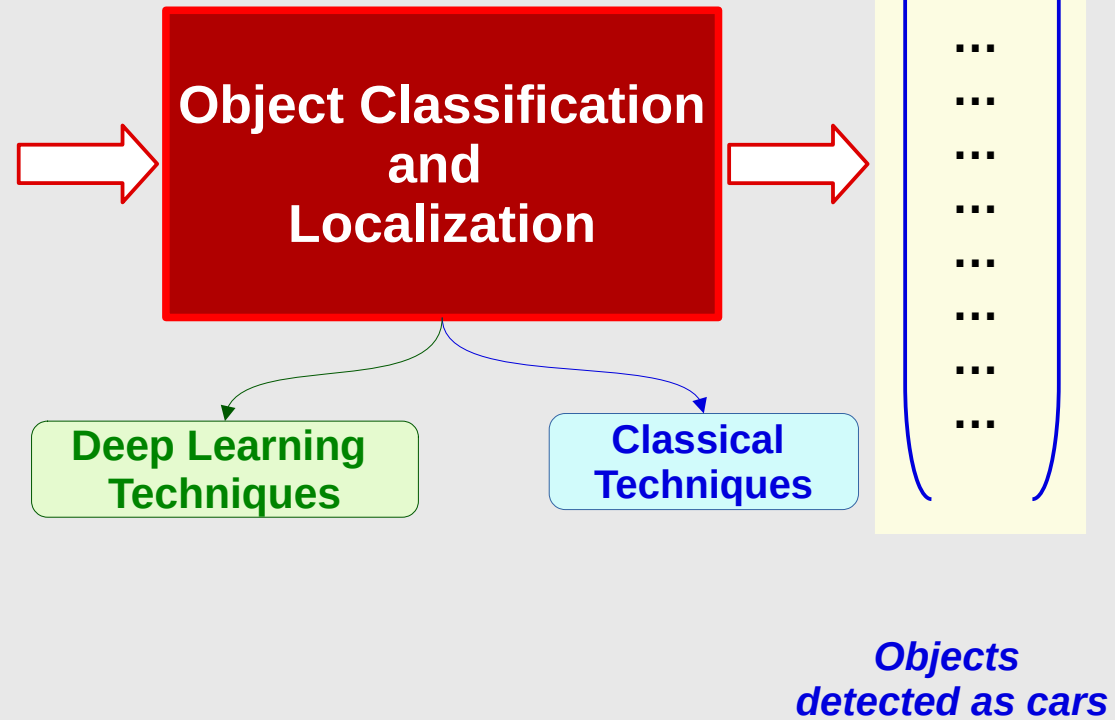
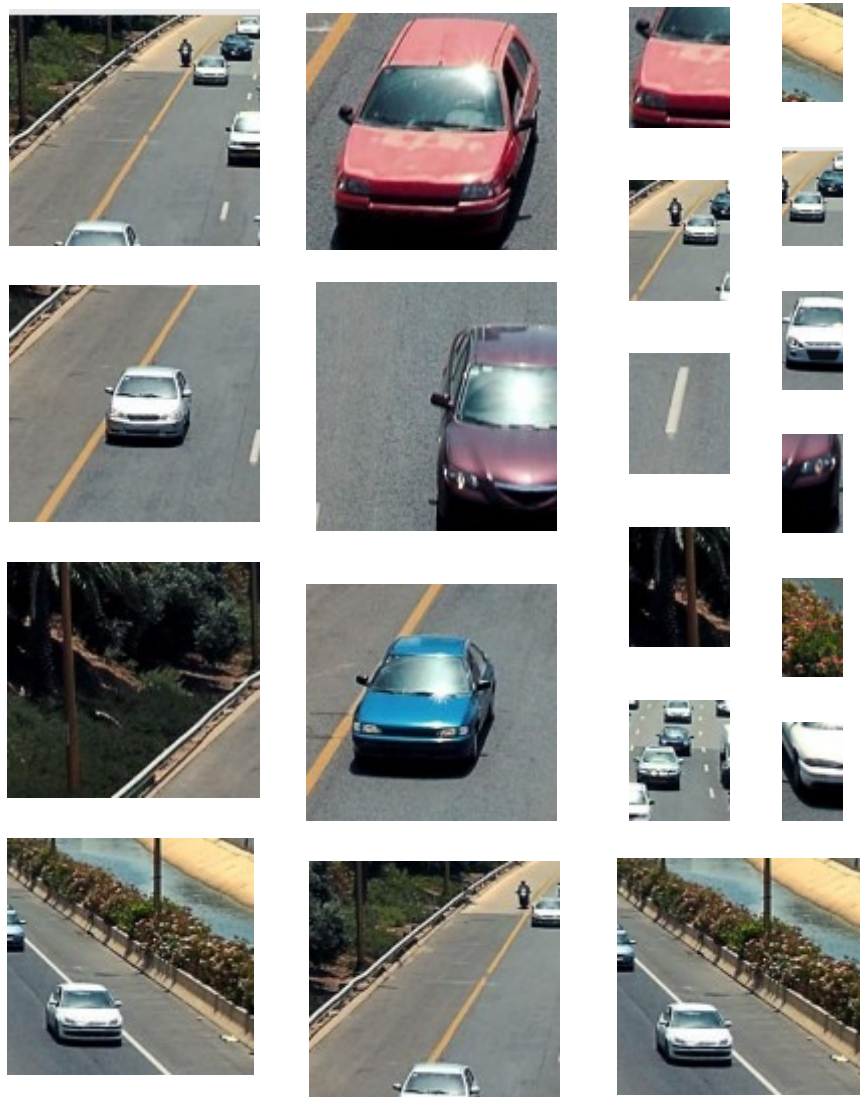
Images Picked from ALL windows sizes



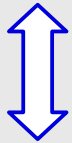
**Object
Classification
(One by One)**



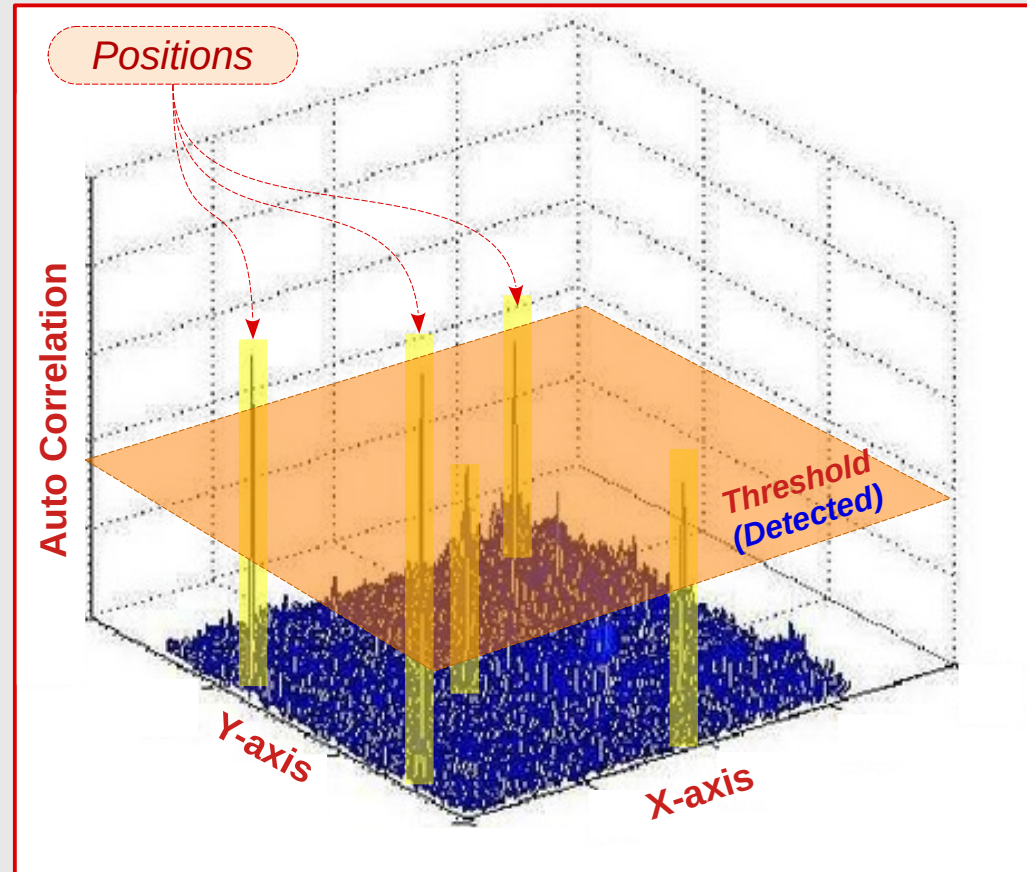
**Objects
detected as cars**



[1] Template Matching using Auto Correlation

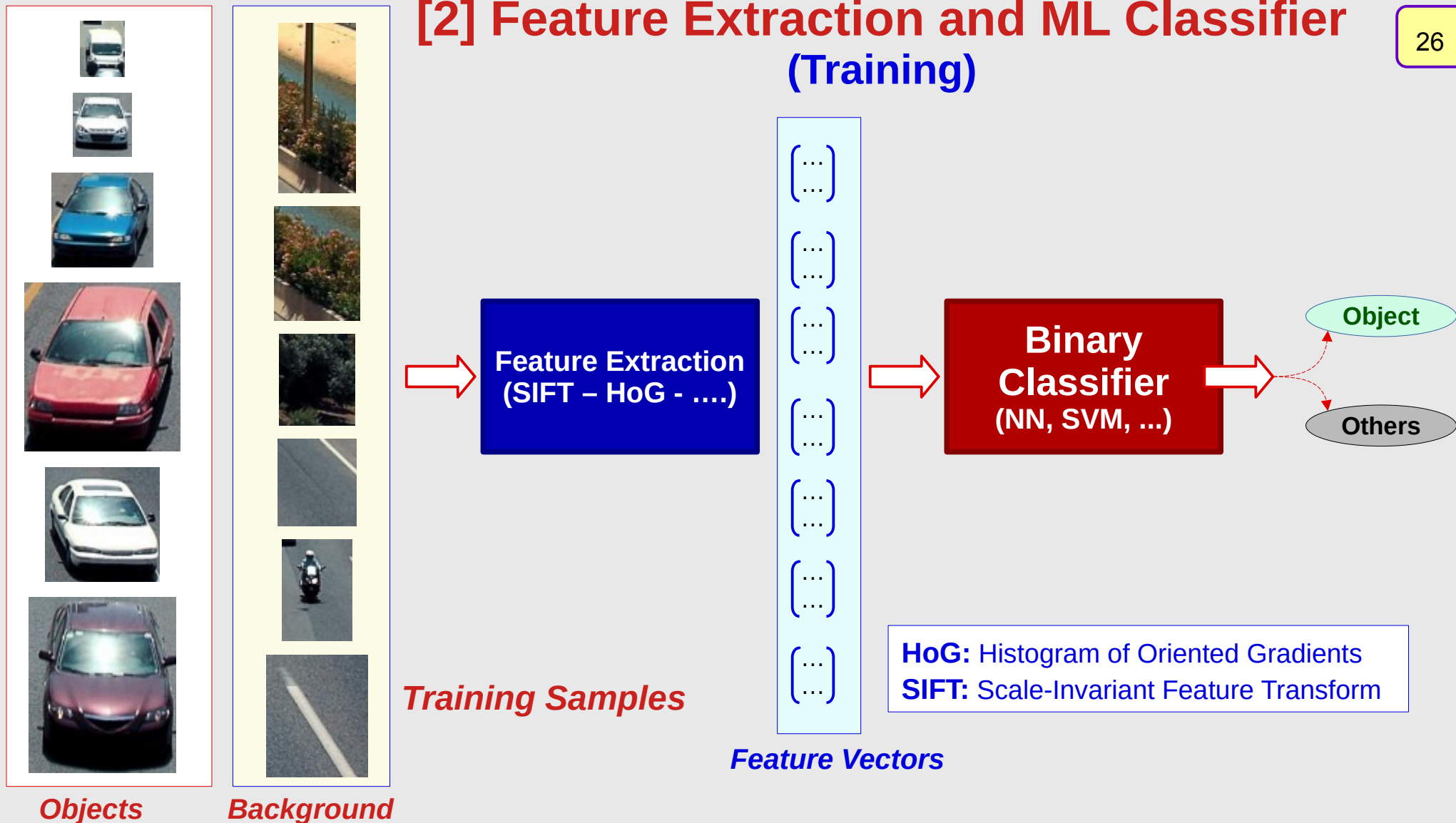


Use different
images sizes



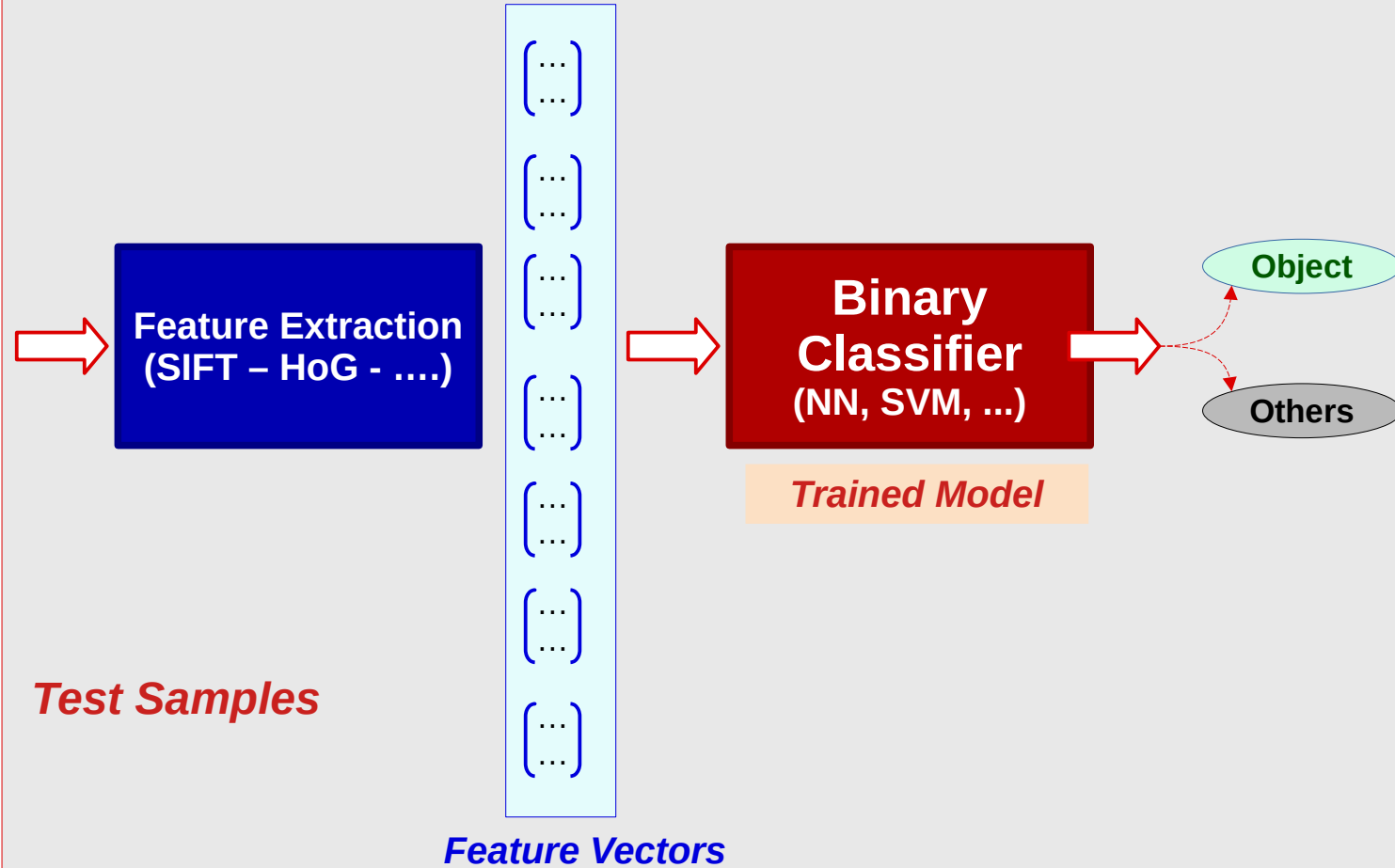
[2] Feature Extraction and ML Classifier (Training)

26



[2] Feature Extraction and ML Classifier (Inference)

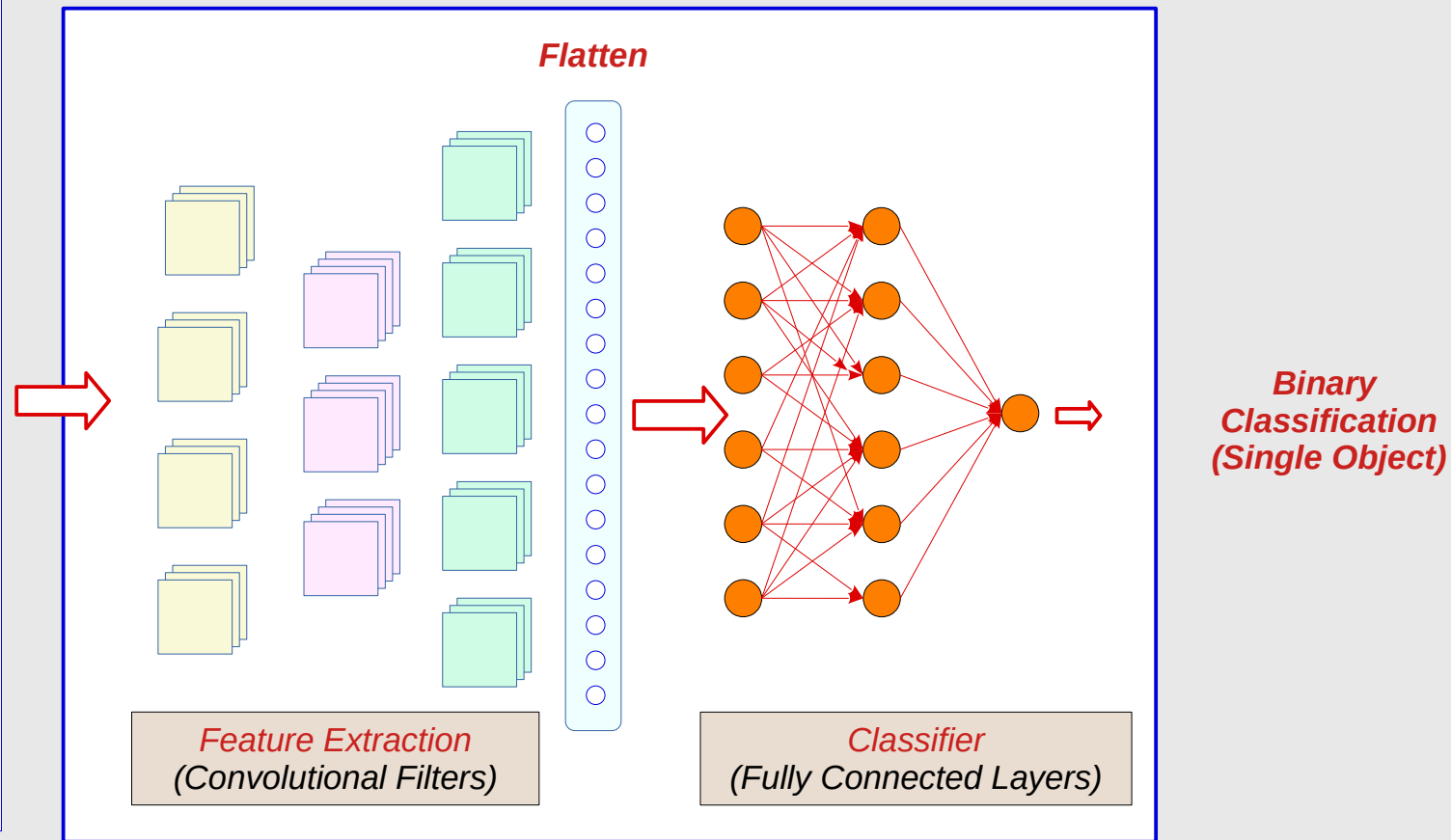
27



Images from Sliding Window at position (x,y)

[3] Convolutional Neural Network (Training - Single Object)

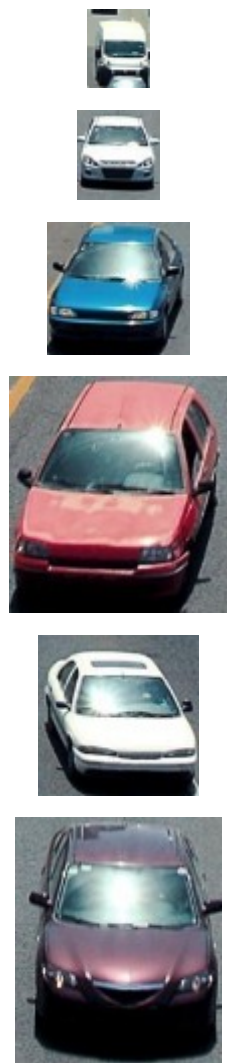
CNN



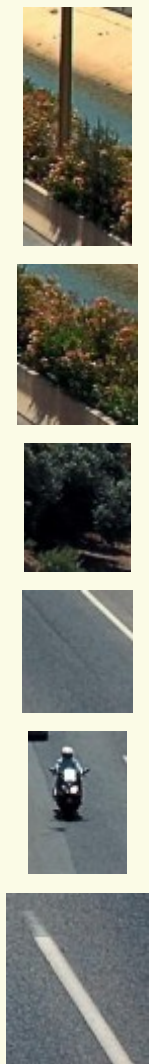
Objects

Background

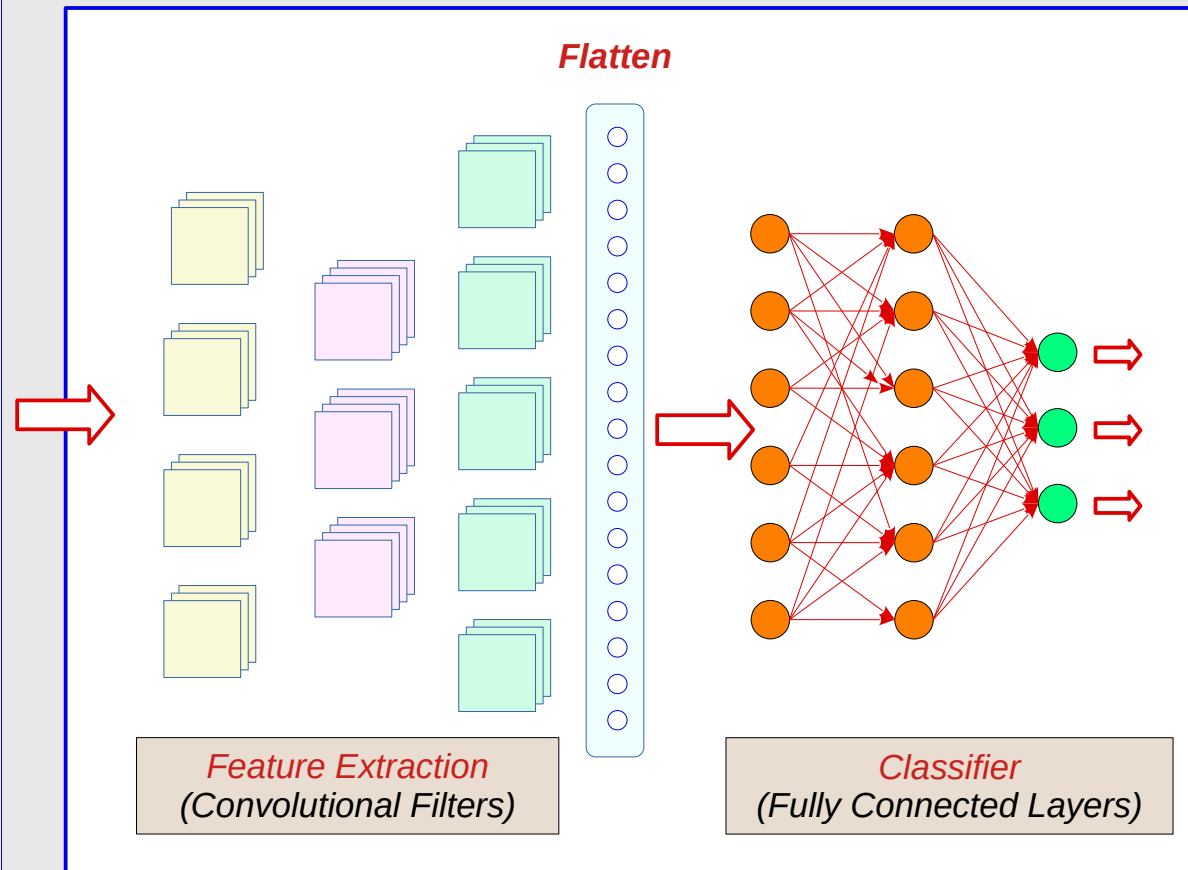
[3] Convolutional Neural Network (Training - Multiple Objects) CNN



Objects



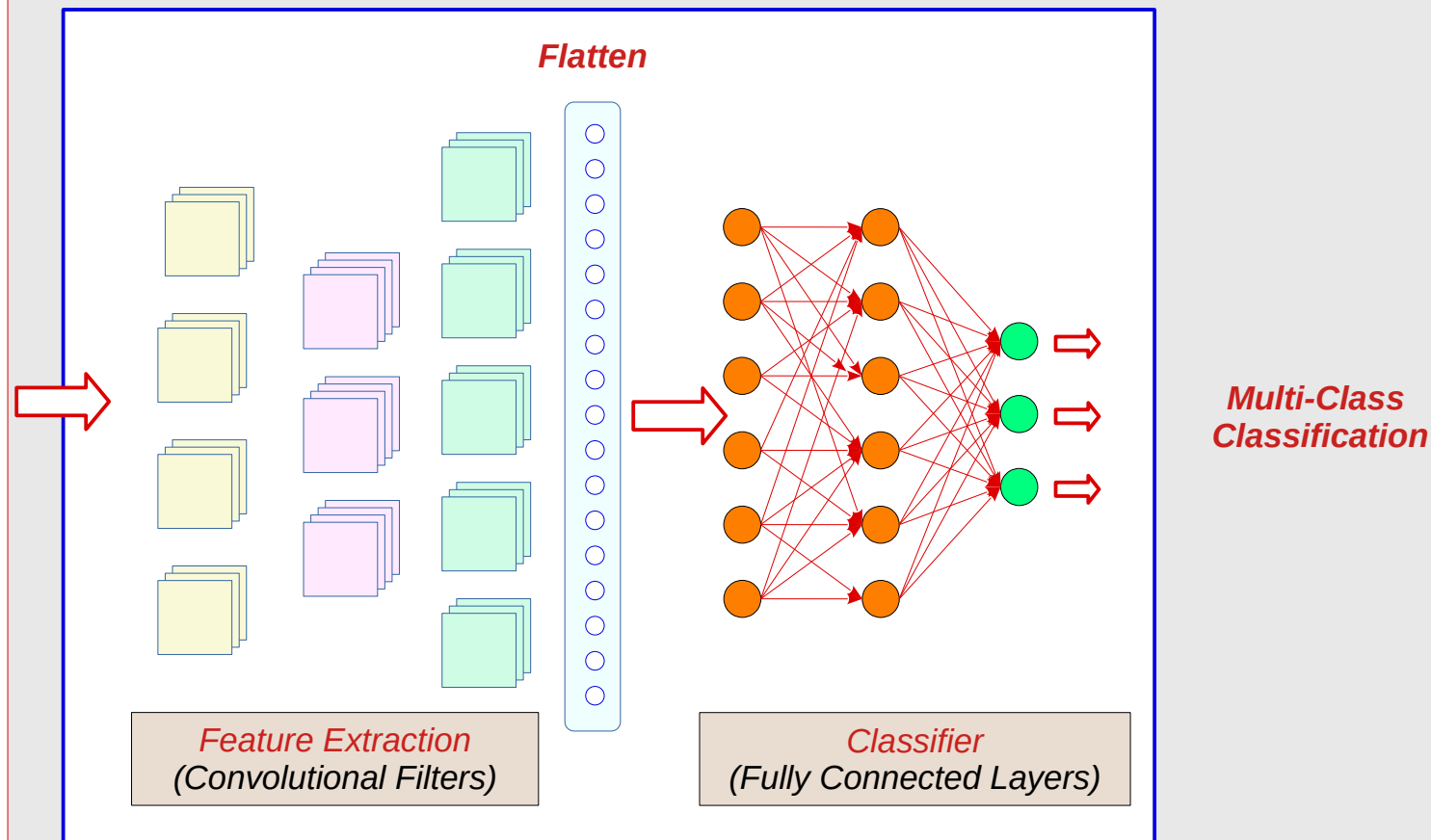
Background



[3] Convolutional Neural Network (Inference - Multiple Objects)

Test Samples

CNN



Images from Sliding Window at position (x,y)

Speeding Up Techniques

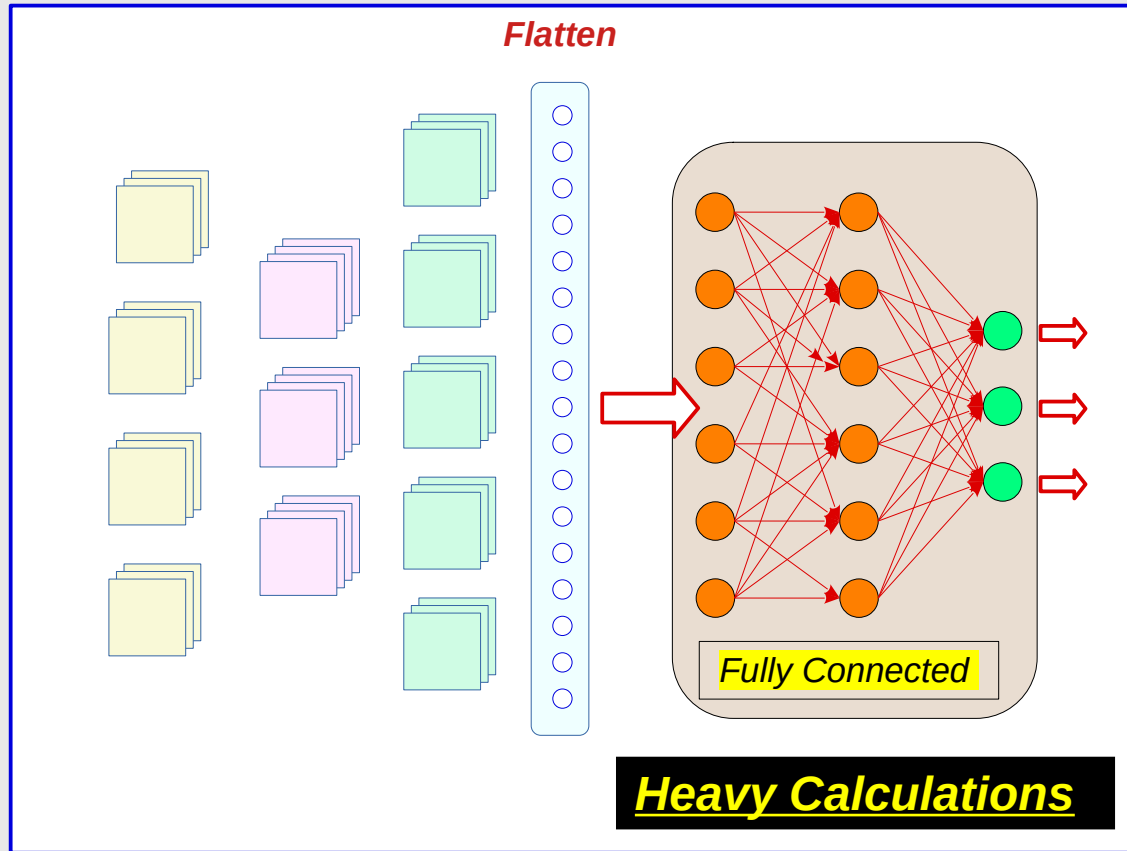
Technique [1]

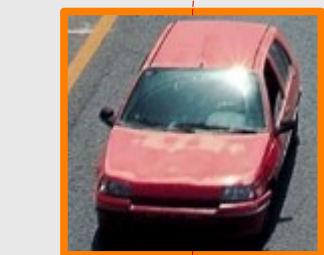
Technique [2]

Technique [3]

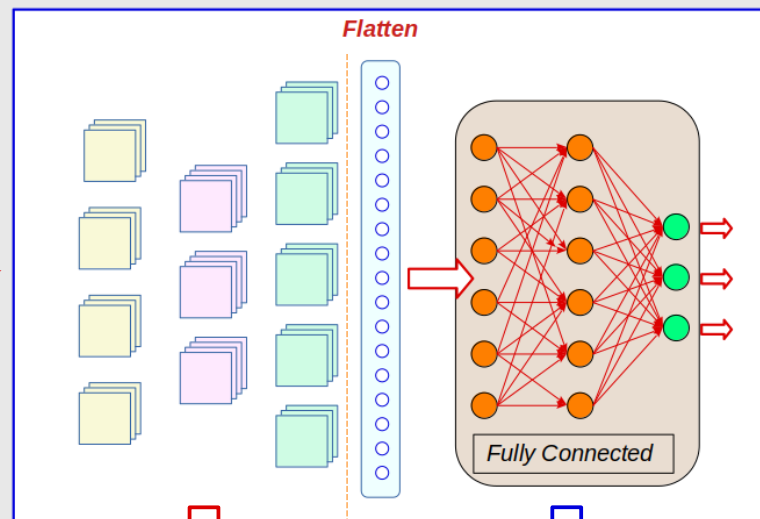
[1] Implementation of Fully Connected Layers with Convolutional Layers

32



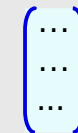
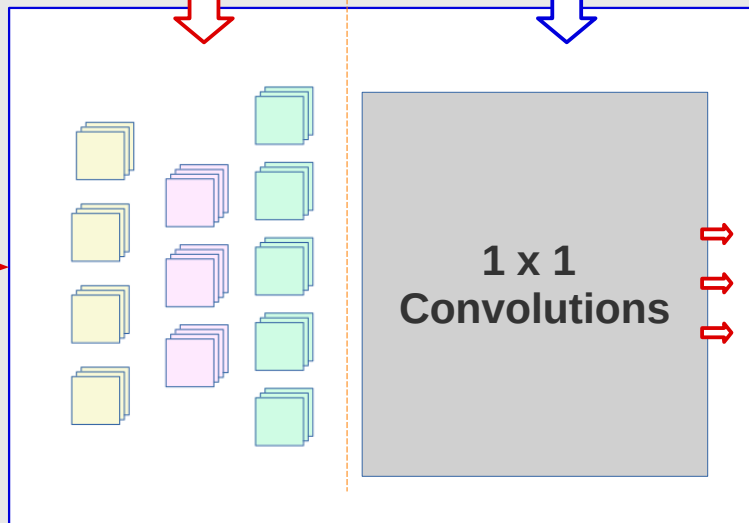


Same Input
dimension



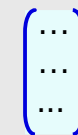
Same

Replace



Replace
Fully Connected Layers
with
1x1 Convolutions

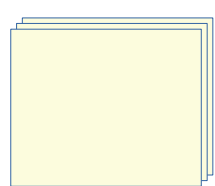
Required:
Same Output Size



Example: Implementation of FC Layers with Conv. Layers

34

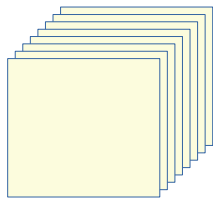
Stride=1 , No Padding



24x24x3



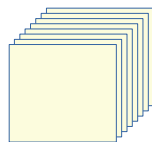
10 Filters
(5x5x3)



20x20x10



Max-Pool
2x2



10x10x10

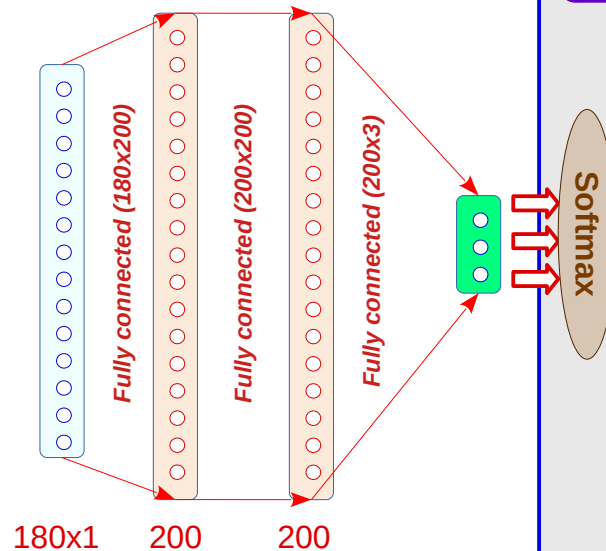


5 Filters
(5x5x10)



6x6x5

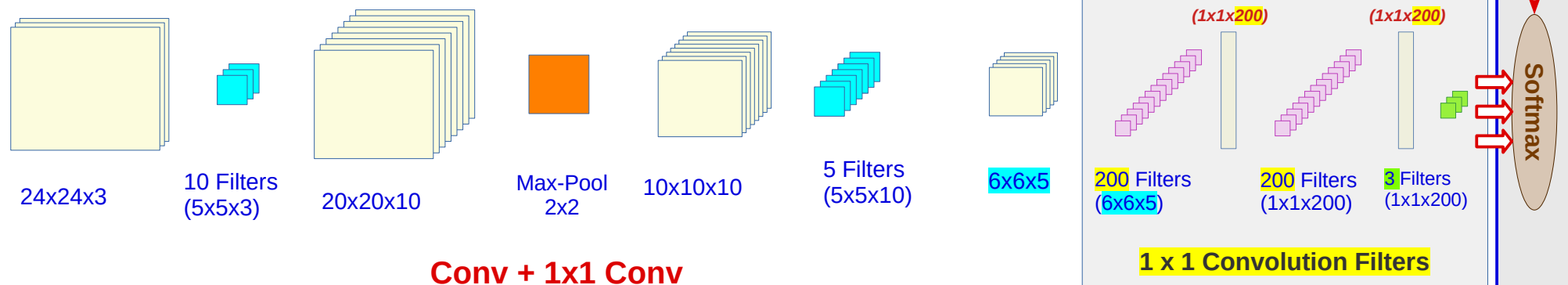
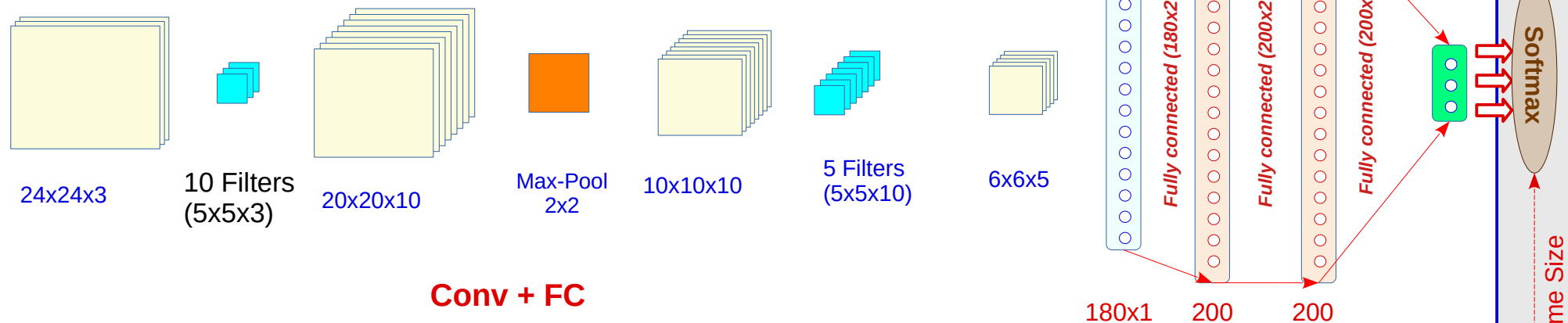
Conv + FC



Softmax

Example: Implementation of FC Layers with Conv. Layers

35



[2] Apply CNN for ALL Sliding windows images at Once

36



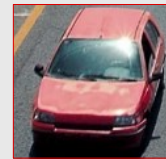
100 x 100 image

Let:
Sliding Window Size = **21x21**

21 x 21

Scan the **100x100** image
using **21x21** sliding window
With Stride=1
No Padding

Results in $80 \times 80 = 6400$ images



21x21

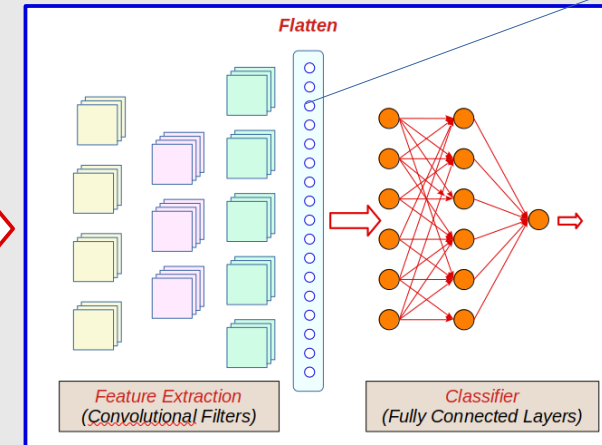


Image
Feature

Classify 6400 images (**one by one**)

[2] Apply CNN for ALL Sliding windows images at Once

37



100 x 100 image



21x21

Scan the 100x100 image
using 21x21 sliding window
With Stride 1
No padding

results in $80 \times 80 = 6400$ images



21x21

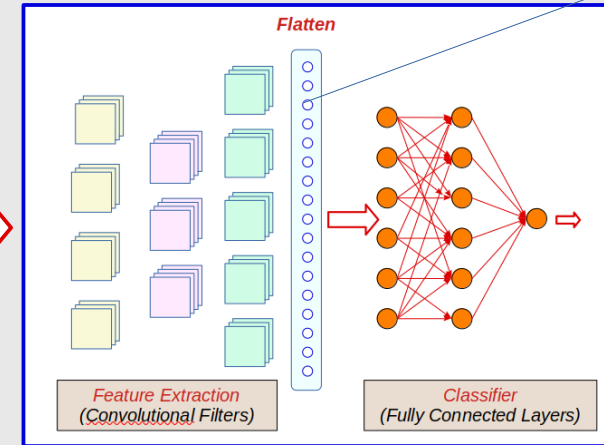


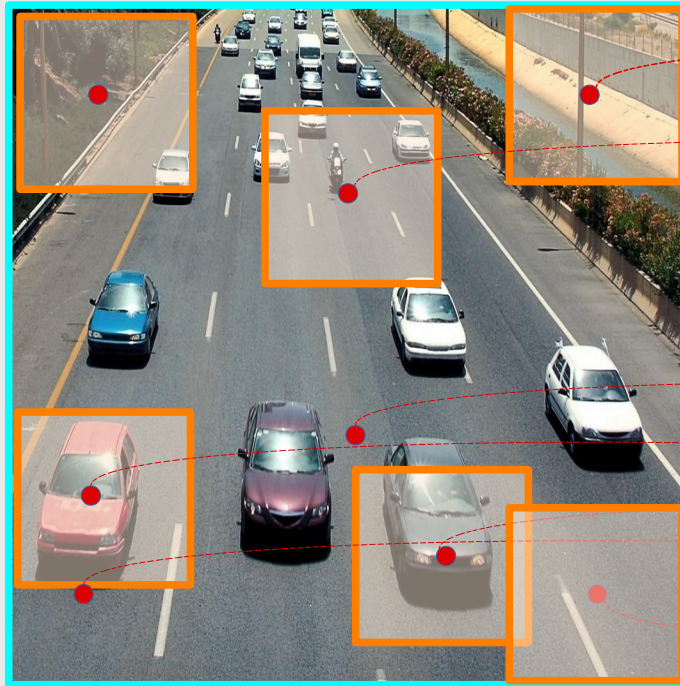
Image
Feature

Let:
Sliding Window Size = **21x21**

Classify 6400 images (**one by one**)

[2] Apply CNN for ALL Sliding windows images at Once

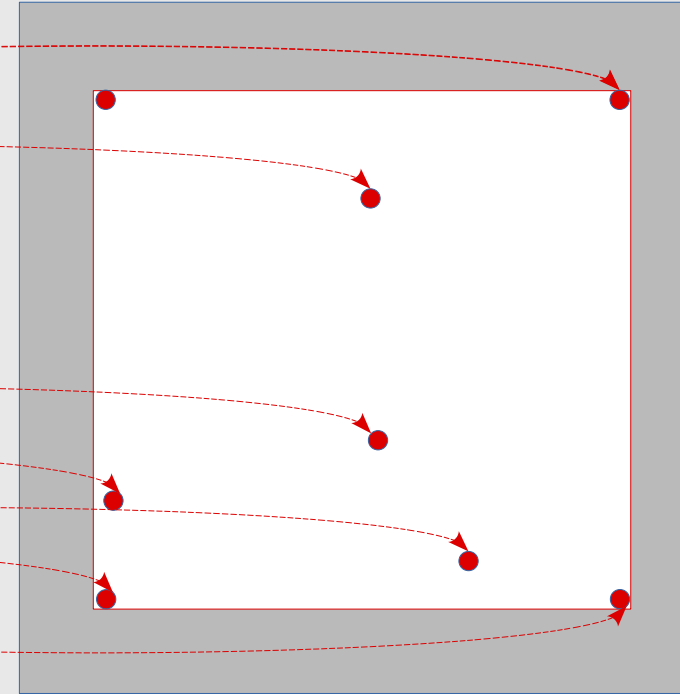
38



100x100

Apply Filter of size **21x21**
with *stride= 1, no padding*

21x21
Filter

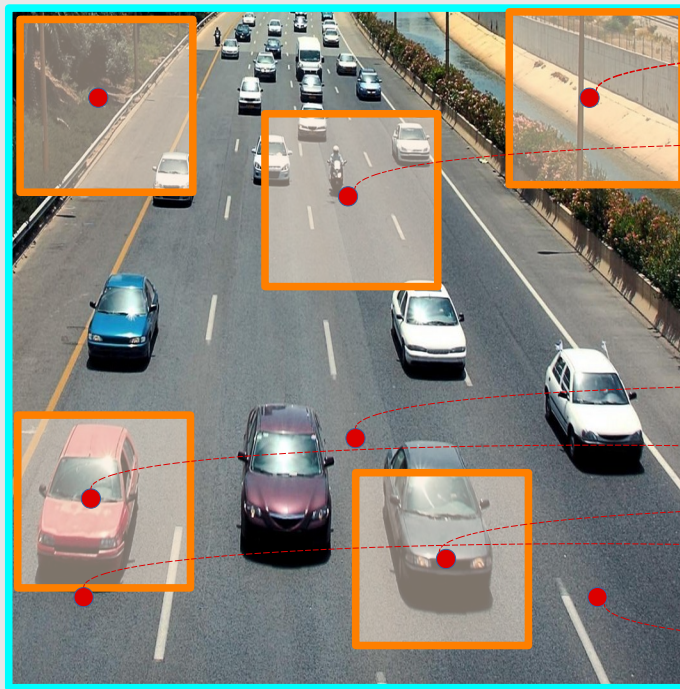


80 x 80

Each output represents value of applying
the filter on the underneath 21x21 image

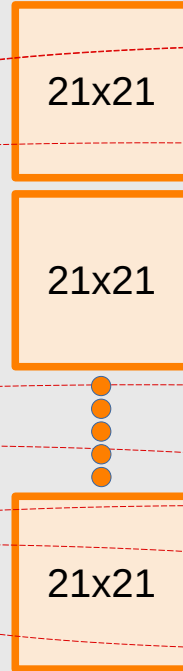
[2] Apply CNN for ALL Sliding windows images at Once

39

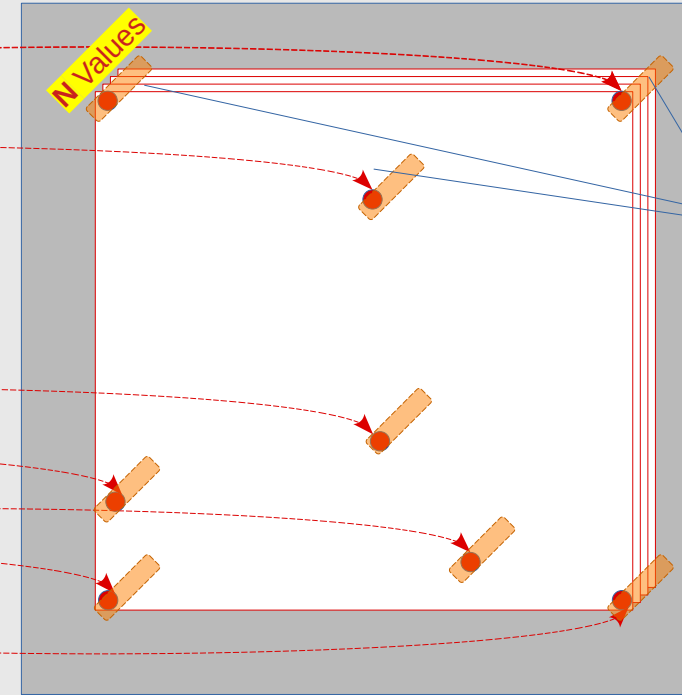


100x100

Apply Filter of size 21x21
with stride 1, no padding



N filters
of size
(21x21x3)

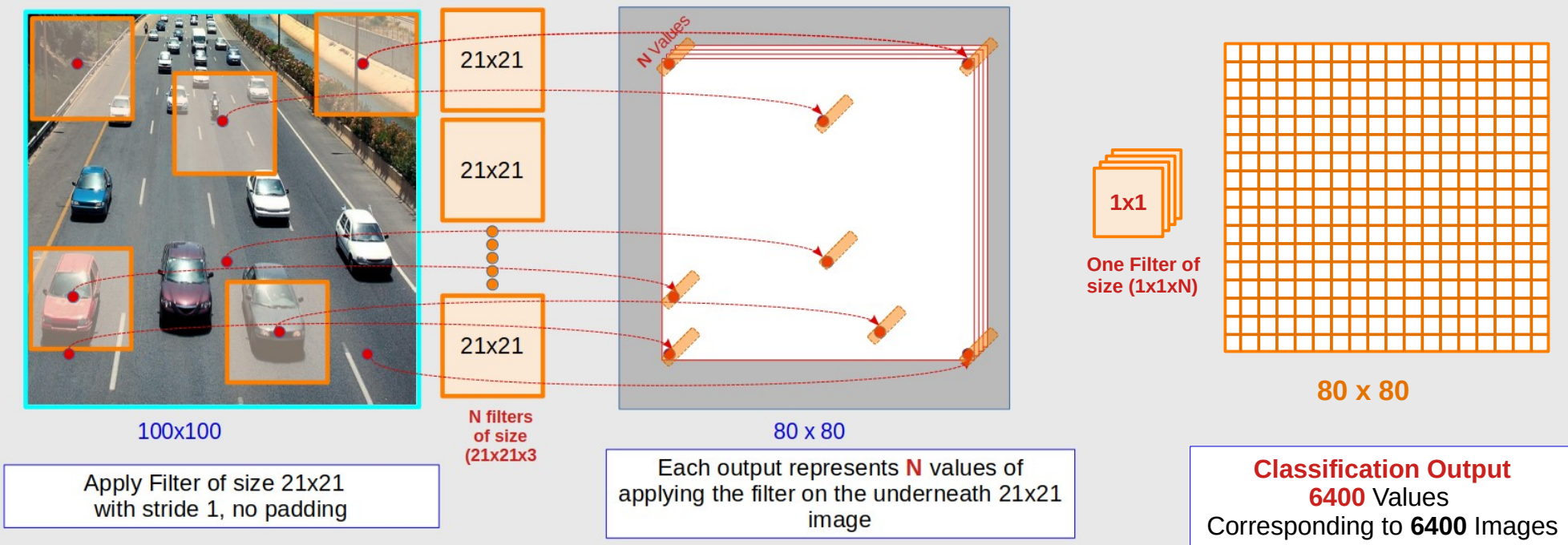


80 x 80

Each output represents **N** values of
applying the **N** filters on the underneath
21x21 image

[2] Apply CNN for ALL Sliding windows images at Once

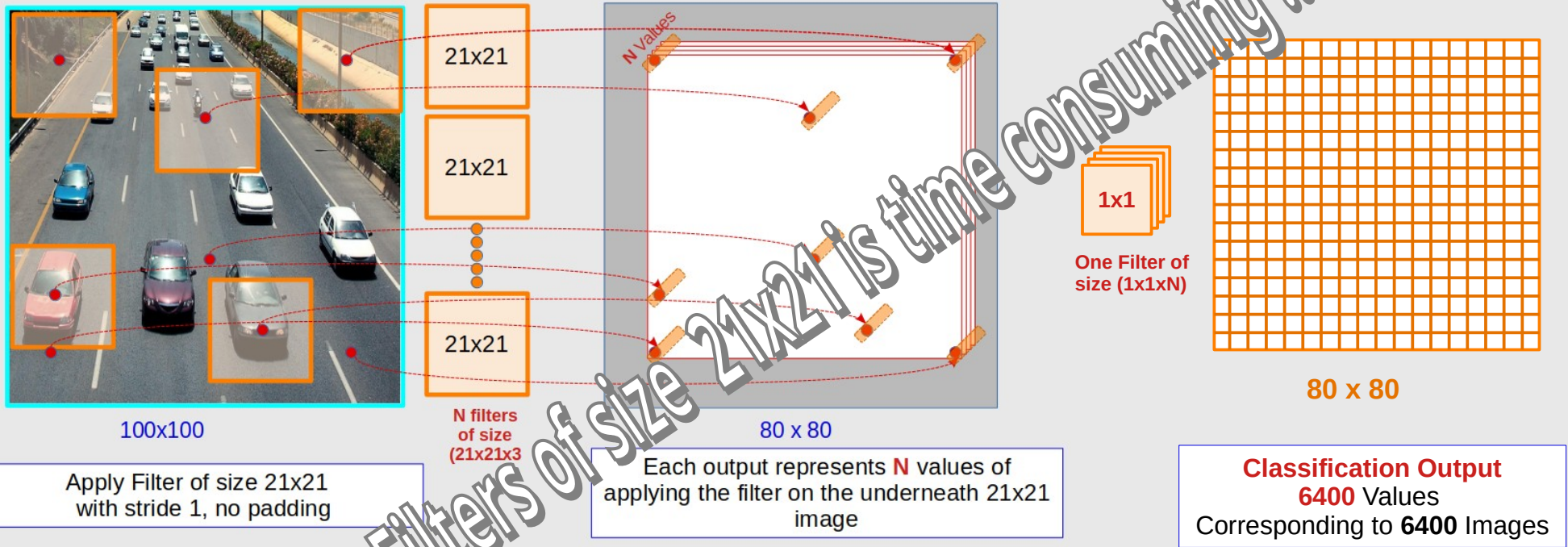
40



Use shared Calculations to parallelize classification of 6400 images (one shot)

[2] Apply CNN for ALL Sliding windows images at Once

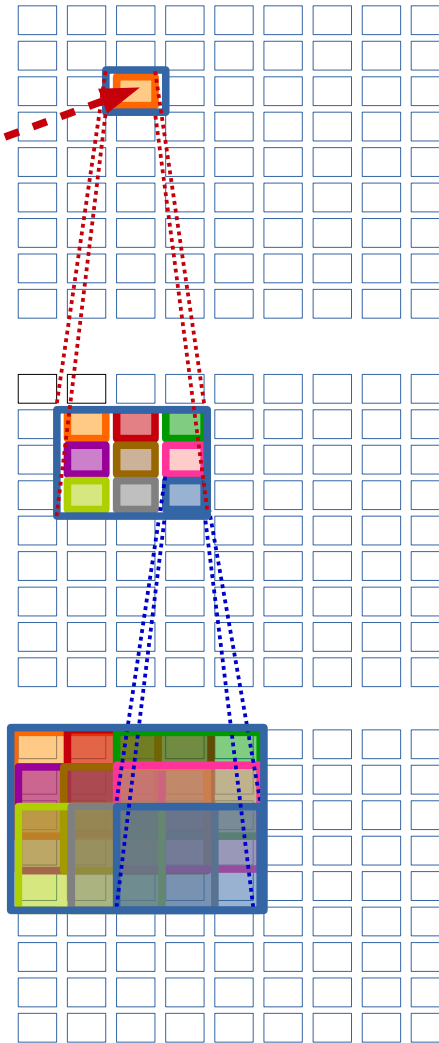
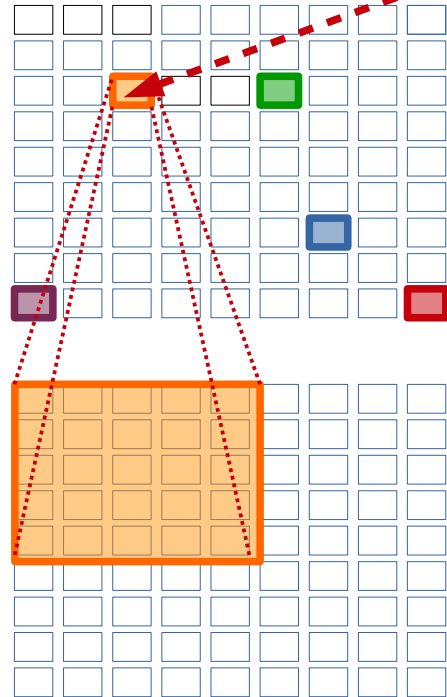
41



Use shared calculations to parallelize classification of 6400 images (one shot)

Review: Filters Factorization

5 x 5 Convolution
Using two Layers 3 x 3 Convolution



Output Representing
5x5 sub-image

Calculations are
reduced to **72%**

[3] Factorize the 21x21 filter using 5x5 filters

43



100 x 100 x 3

5x5x3

N Filters

96 x 96 x N

5x5xN

N Filters

92x92 x N

5x5xN

N Filters

88 x 88 X N

5x5xN

N Filters

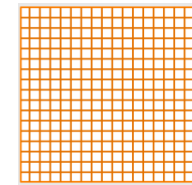
84 x 84 x N

N Filters

5x5xN

80 x 80 x N

Five 5 x 5 filters is equivalent to One 21x21 filter



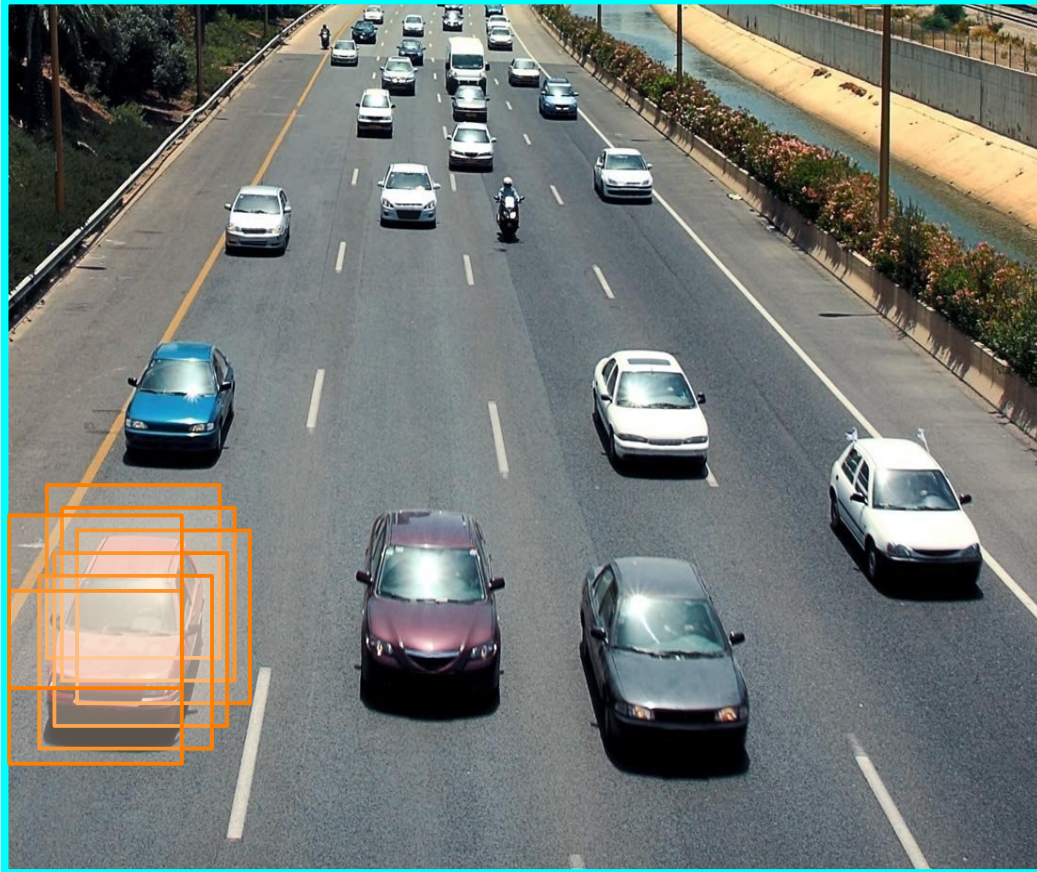
80 x 80 x 1

1x1x 1

1600 Classification Output

Problem of OVERLAPPED Detections

44



**Same object
can be detected with
Multiple windows !!!**

Use
Non-Maxima Suppression
Algorithm
To resolve overlapping problem

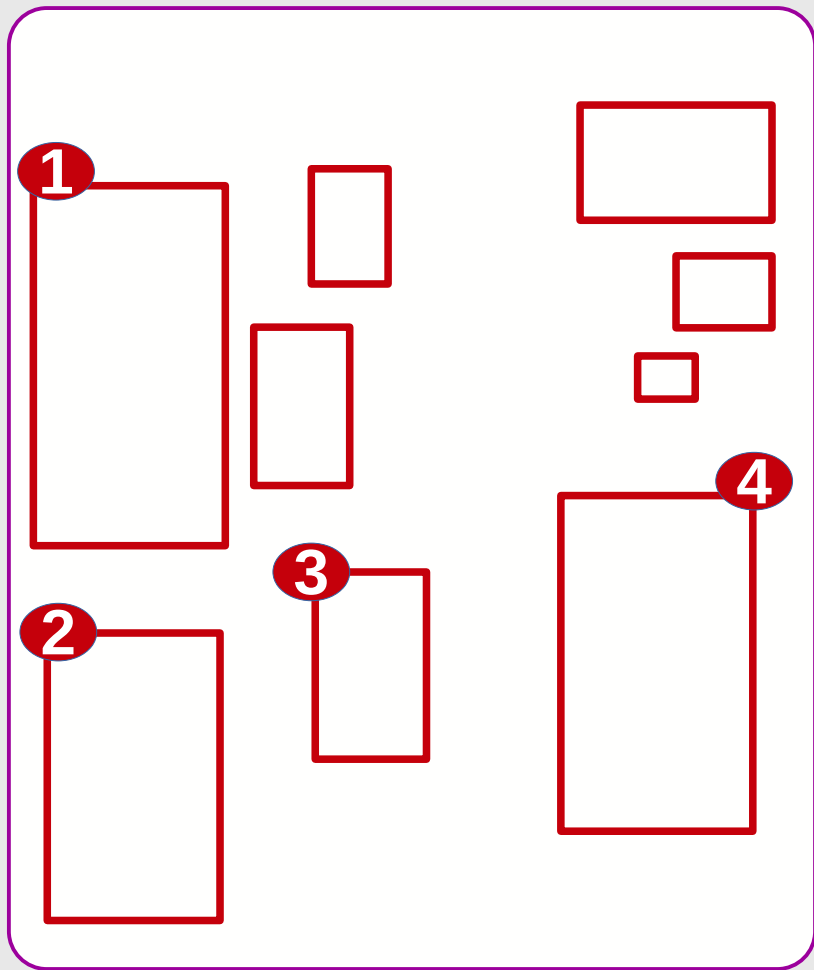
Sliding Window Technique

Detection + Localization

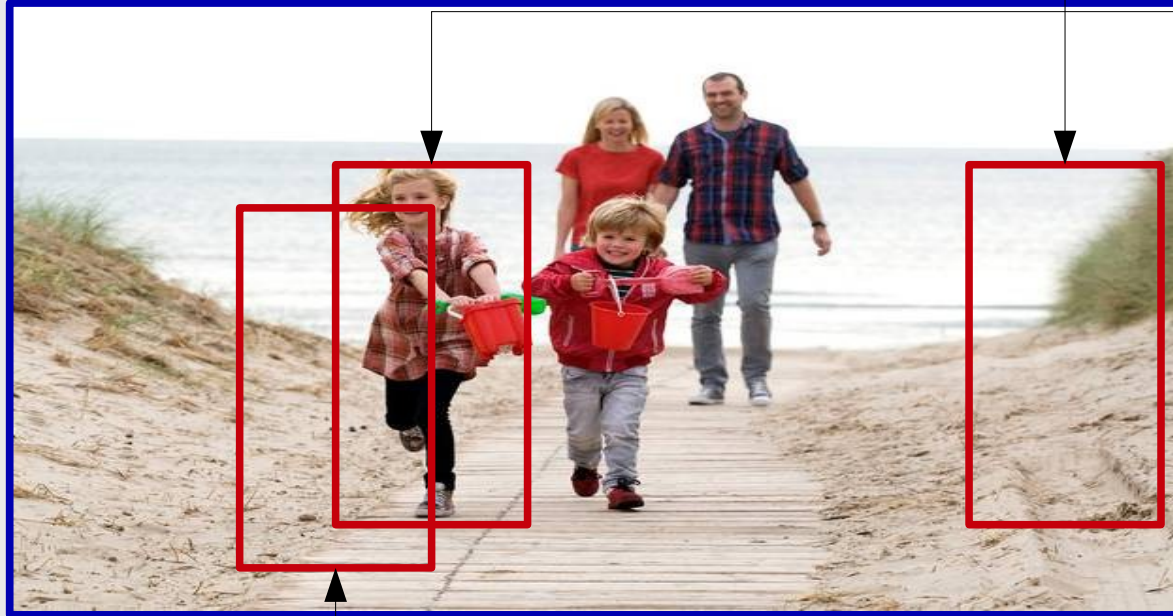


Different Windows Sizes

Scan Multiple Times



Sliding Window Technique



Detect Three Classes

C_1 Person

C_2 Cat

C_3 Book

Final Filter is 1x1x8

(to detect 3 classes)

1
0.3
0.1
0.6
0.9
1
0
0

0
X
X
X
X
X
X
X

0.7
0.2
0.2
0.6
0.9
1
0
0

P
C
B_x
B_y
B_w
B_h
C₁
C₂
C₃

Sliding Window Technique

- **Very Very Slow (Computational Cost)**
- **Pass through the Image Many Times with Different Sizes**
- **If Stride is large, (loss of precision in localization)**